

Introduction to Computer Programming

Prof. Alessio Martino

`amartino@luiss.it`

Department of Business and Management
Viale Romania 32, 00197 Rome



A.Y. 2024-2025

Outline

- 1 Iteration
- 2 `while` loops
- 3 `for` loops
 - the `range()` function
- 4 `for` vs. `while` loops
- 5 More on `for` loops
- 6 Nested Loops
- 7 `break` statement
- 8 References
- 9 Acknowledgments

Iteration

- **Iteration** is the process where a set of instructions (or statements) is executed repeatedly for a specified number of time or until a condition is met.
- Like branching, these statements also **alter the control flow** of the program.

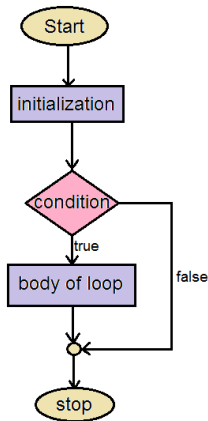


Figure 1: Anatomy of a looping mechanism.

while loops

```
while <condition>:  
    statement  
    statement  
    ...  
    statement
```

- <condition> evaluates to a Boolean (True or False)
- 1 if <condition> is True, then execute all statements inside the while block
 - 2 evaluate <condition> again
 - 3 repeat until <condition> is False

Example:

```
n = input("You are in the Lost Forest. Go Left or Right?: ")
while n == "Right":
    n = input("You are in the Lost Forest. Go Left or Right?: ")
print("You are out of the Lost Forest.")
```

for loops

```
for <variable> in range(<number>):
    statement
    statement
    ...
    statement
```

- each time through the loop, `<variable>` takes some value
- `<variable>` starts from the smallest value (usually, 1)
- next time, `<variable> = <variable> + 1`

The `range(start,stop,step)` function returns a sequence of numbers, starting from `start`, and increments by `step`, and stops at `stop-1`.

Keep in mind the following caveats:

- `stop` is mandatory
- `step` is optional: if not provided, it defaults to 1
- `start` is optional: if not provided, it defaults to 0

Examples:

```
mysum = 0
for i in range(7, 10):
    mysum = mysum + i
print(mysum)
```

```
mysum = 0
for i in range(5, 11, 2):
    mysum = mysum + i
print(mysum)
```

for vs. while loops

Iterate through sequence of numbers in $[0, 5)$:

```
# with while loop
```

```
n = 0
```

```
while n < 5:
```

```
    print(n)
```

```
    n = n + 1
```

```
# with for loop
```

```
for n in range(5):
```

```
    print(n)
```


for loops:

- **known** number of iterations
- can **end early** via **break**
- (natively) uses a **counter**
- you **can rewrite** a **for** loop using a **while** loop

while loops:

- **unbounded** number of iterations
- can **end early** via **break**
- you can use a **counter**, but you must initialise it before the loop and increment it inside the loop
- you **may not be able to rewrite** a **while** loop using a **for** loop

More on for loops

`for` loops can be used to iterate over other data structures (e.g., lists or dictionaries¹).

Example: print all functions in the `math` module that contain the word 'log'

```
import math
for f in dir(math):
    if 'log' in f:
        print(f)
print("Done!")
```

Recall

`dir()` returns a list.

¹We will see lists and dictionaries later in the course

Nested Loops

We know that branching (i.e., if) statements can be arbitrarily nested. The same holds for loops.

Example:

```
mysum = 0
```

```
for i in range(3):  
    for x in range(2):  
        mysum += i
```

```
print(mysum)
```

Recall

`mysum += i` is the same as `mysum = mysum + i`

break statement

The **break** statement:

- immediately exits whatever loop it is in
- all further statements in the block are skipped
- exits only the innermost loop
- usually triggered at given condition

```
while <condition_1>:
    while <condition_2>:
        <statement_a>
        break
    <statement_b>
<statement_c>
```

Example:

```
mysum = 0

for i in range(5, 11, 2):
    mysum += i
    if mysum == 5:
        break
    mysum += i

print(mysum)
```

What's going on here?

```
mysum = 0
```

```
for i in range(30):
    for n in range(20):
        if n > 2:
            break
        mysum += i
    if i>1:
        break

print(mysum)
```

What's going on here?

References

- Guttag, J. V. (2013). *Introduction to Computation and Programming using Python (Revised and Expanded Edition)*. MIT Press. [Sections 2.4, 3.2]
- Downey, A. (2015). *Think Python*. Green Tea Press ed. 2nd. [Sections 4.2, 7.3, 7.4]

Acknowledgments

- <https://www.codinggeek.com/tutorials/c-programming/for-loop-in-c-programming-language-iteration-statements/>
- <https://docs.python.org/3/library/random.html>